TransportTN

Plateforme Web de Réservation de Transport

Taxi privé · Taxi collectif · Bus interurbain

CAHIER DES CHARGES — Version Web 2.0

Projet académique — Tunisie

Stack technique : React (Vite) · Tailwind CSS · Spring Boot 3.2 · Java 17 · Firebase · MySQL 8 · Docker

1. Présentation générale du projet

TransportTN est une plateforme web responsive destinée à moderniser la réservation de transports en Tunisie. Depuis un navigateur sur ordinateur, tablette ou smartphone, les utilisateurs peuvent réserver un taxi privé, réserver une ou plusieurs places dans un taxi collectif, ou acheter un ticket de bus interurbain. La plateforme propose des confirmations, des notifications et, pour les courses taxi, un suivi de position en temps réel lorsque le navigateur et les autorisations le permettent.

1.1 Contexte et motivation

Le secteur du transport présente des difficultés de visibilité sur les disponibilités, de planification des déplacements et de conservation des tickets. TransportTN centralise ces opérations dans une interface accessible sans installation d'application mobile.

1.2 Objectifs

• Créer une plateforme web unifiée couvrant taxi privé, taxi collectif et bus interurbain.

• Fournir une interface responsive, simple et utilisable sur les navigateurs modernes.

• Sécuriser les sessions et les opérations avec Firebase Authentication (OTP) et des jetons JWT.

• Afficher le suivi de position du chauffeur en temps réel lorsque le partage de position est autorisé.

• Générer un QR code unique pour chaque ticket de bus confirmé.

• Permettre une évolution progressive vers d'autres villes et régions de Tunisie.

2. Périmètre fonctionnel

2.1 Modules applicatifs

Module	Description	Priorité
Authentification	Connexion par numéro tunisien et code OTP via Firebase Auth.	P1 — Essentiel
Réservation taxi	Saisie départ/arrivée sur carte, estimation du prix et recherche d'un chauffeur disponible.	P1 — Essentiel
Taxi collectif	Choix d'une ligne, nombre de places et vérification de disponibilité.	P1 — Essentiel
Tickets bus	Choix ligne/date/quantité, confirmation et génération d'un QR code.	P1 — Essentiel
Suivi temps réel	Affichage de la position du chauffeur sur la carte web via Firestore.	P1 — Essentiel
Notifications	Notifications web compatibles avec le navigateur et autorisées par l'utilisateur; état consultable dans l'interface.	P1 — Essentiel
Historique	Liste des réservations et tickets passés.	P2 — Important
Profil	Gestion des informations personnelles et préférences.	P2 — Important
Administration	Gestion des lignes et consultation de statistiques.	P3 — Optionnel

2.2 Réservation taxi privé

• Saisie des adresses de départ et d'arrivée via Google Maps JavaScript API.

• Affichage des marqueurs et calcul de distance pour estimer le prix.

• Choix du mode de paiement disponible : espèces ou carte, selon l'intégration retenue.

• Recherche du chauffeur disponible le plus proche et notification des parties.

• Suivi de la position sur la carte web; le partage de géolocalisation nécessite une autorisation du navigateur.

• Annulation possible selon les règles métier.

2.3 Taxi collectif

• Consultation des lignes, itinéraires, horaires, tarifs et places restantes.

• Sélection de 1 à 4 places selon la disponibilité.

• Vérification côté serveur avant validation et envoi d'une confirmation.

2.4 Bus interurbain

• Catalogue des lignes avec départ, destination, horaires et tarif.

• Sélection de la date de voyage et du nombre de tickets.

• Génération d'un QR code unique après confirmation; consultation depuis l'espace personnel web.

3. Architecture technique

La solution adopte une architecture web en couches. Le frontend React communique avec le backend Spring Boot via HTTPS et API REST. Firebase assure l'authentification OTP, le suivi temps réel et les notifications compatibles web. MySQL conserve les données métier.

Couche	Technologie	Rôle
Frontend web	React + Vite, Tailwind CSS, React Router	Pages responsive, formulaires, navigation, affichage des réservations.
API backend	Spring Boot 3.2, Java 17, Maven	Règles métier, API REST, validation et autorisation.
Base de données	MySQL 8, Spring Data JPA / Hibernate	Persistance des utilisateurs, réservations, lignes et tickets.
Authentification	Firebase Authentication OTP + JWT + Spring Security	Connexion et contrôle d'accès par rôle.
Temps réel	Firebase Firestore	Transmission de la position et mise à jour de la carte.
Notifications	Firebase Cloud Messaging pour le Web	Alertes navigateur après autorisation et selon compatibilité.
Déploiement	Docker, Nginx, HTTPS, VPS Ubuntu, GitHub Actions	Hébergement sécurisé, automatisation des tests et déploiement.

3.1 Schéma de base de données

Le modèle initial conserve six tables métier : users (clients, chauffeurs et administrateurs), drivers (profil et disponibilité du chauffeur), reservations (réservations taxi), lignes (lignes de bus et taxi collectif), tickets (tickets de bus avec QR code) et reservation_seats (places réservées dans un taxi collectif). Les champs exacts seront définis dans le schéma SQL.

3.2 Sécurité

• Authentification OTP par Firebase pour les numéros +216, si cette restriction est conservée.

• Jetons JWT signés et vérifiés côté backend; expiration et renouvellement à définir selon la politique de sécurité.

• Rôles CLIENT, DRIVER et ADMIN avec autorisations par endpoint.

• CORS limité aux domaines autorisés; HTTPS obligatoire en production.

• Les clés privées et fichiers de service Firebase ne doivent jamais être exposés dans le code frontend ou déposés dans Git.

4. Interfaces utilisateur web (UI/UX)

4.1 Parcours principal

Étape	Page web	Action
1	Accueil	Présentation des services et accès à la connexion.
2	Connexion	Saisie du numéro +216 et demande du code OTP.
3	Vérification OTP	Validation du code et ouverture de session.
4	Tableau de bord	Choix Taxi, Collectif ou Bus.
5	Formulaire de réservation	Saisie des détails, ligne, date, quantité ou adresses.
6	Confirmation	Récapitulatif et validation de la demande.
7	Suivi / Mes tickets	Carte GPS web ou affichage du QR code.

4.2 Exigences UI

• Interface responsive avec Tailwind CSS, utilisable sur mobile, tablette et ordinateur.

• Navigation accessible : Accueil, Taxi, Collectif, Bus, Profil et Administration selon le rôle.

• Indicateurs de chargement et messages d'erreur compréhensibles en français.

• Formulaires validés côté frontend et obligatoirement revérifiés côté backend.

• Prise en charge de Chrome, Firefox, Edge et Safari récents; tests sur plusieurs tailles d'écran.

5. Spécifications de l'API REST

L'API REST est exposée par Spring Boot. En développement, elle peut utiliser le port 8080; en production, elle est publiée en HTTPS derrière Nginx. Les routes protégées nécessitent une authentification et des droits adaptés.

Méthode	Endpoint	Description
POST	/api/auth/login	Échange du jeton Firebase contre un JWT applicatif.
GET	/api/auth/me	Profil de l'utilisateur connecté.
POST	/api/reservations/taxi	Créer une réservation taxi.
POST	/api/reservations/collectif	Réserver des places en taxi collectif.
POST	/api/reservations/bus	Acheter un ou plusieurs tickets de bus.
GET	/api/reservations/history	Consulter l'historique des réservations.
PUT	/api/reservations/{id}/status	Mettre à jour le statut d'une réservation.
GET	/api/reservations/lignes	Lister les lignes disponibles.
GET	/api/reservations/tickets	Lister les tickets du client.
PUT	/api/driver/location	Mettre à jour la position du chauffeur autorisée par le navigateur.
POST	/api/admin/lignes	Créer une ligne (rôle ADMIN).

Toutes les réponses utilisent JSON. Les erreurs retournent un code HTTP approprié (400, 401, 403, 404, 500) accompagné d'un message descriptif.

6. Contraintes non fonctionnelles

6.1 Performance et disponibilité

• Objectif de réponse API inférieur à 500 ms pour les requêtes métier, hors latence réseau.

• Pagination des listes et indexation des colonnes fréquemment recherchées.

• Objectif de disponibilité en production : 99,5 %, à confirmer selon l'hébergement.

• Déploiement sur serveur Linux, HTTPS via Nginx et certificat TLS.

• Docker Compose pour reproduire l'environnement; CI/CD avec tests automatisés.

• La géolocalisation web dépend de l'autorisation utilisateur, du navigateur et d'une connexion HTTPS.

6.2 Maintenabilité et qualité

• Frontend organisé en composants, pages, services API et gestion centralisée des erreurs.

• Backend structuré en Controller / Service / Repository / Model.

• Tests unitaires backend avec JUnit 5 et Mockito; tests API et tests de parcours frontend.

• Documentation des endpoints et instructions de lancement dans le README.

7. Planning et livrables

Phase	Contenu	Durée estimée
1 — Mise en place	Configuration React/Vite, Spring Boot, Firebase et MySQL.	1 semaine
2 — Authentification	OTP Firebase côté web, JWT et rôles côté backend.	1 semaine
3 — Backend	Entités JPA, services métier, API REST et validations.	2 semaines
4 — Frontend web	Pages React, navigation, formulaires, carte et espace utilisateur.	2 semaines
5 — Temps réel	Firestore, suivi de position web et notifications navigateur.	1 semaine
6 — Tests et déploiement	Tests, Docker, CI/CD, configuration HTTPS et mise en ligne.	1 semaine

7.1 Livrables attendus

• Code source backend Spring Boot et frontend React dans un dépôt Git.

• Schéma de base de données SQL et diagramme associé.

• Documentation des API et collection de tests API.

• Cahier des charges et présentation PowerPoint mis à jour.

• Guide README pour installer, configurer, tester et déployer l'application web.

• Build frontend de production et configuration de déploiement web.

8. Risques et mesures d'atténuation

Risque	Mesure d'atténuation
Quota Google Maps dépassé	Limiter les appels, mettre en cache les données pertinentes et surveiller l'usage.
Réseau internet instable	Timeouts, messages d'erreur clairs, reprises contrôlées et états de chargement.
Notifications web non autorisées	Demander la permission au bon moment et afficher les notifications dans l'espace web.
Géolocalisation indisponible	Informer l'utilisateur; vérifier HTTPS et la permission du navigateur.
Fuite de JWT ou secrets	HTTPS, durée de vie limitée, stockage prudent des jetons et secrets uniquement côté serveur.
Performance sous charge	Index MySQL, pagination, journalisation et dimensionnement du serveur.

9. Conclusion

TransportTN devient une plateforme web responsive de réservation de transport, accessible sans installation depuis un navigateur. React et Tailwind CSS assurent l'interface; Spring Boot fournit les services métier et l'API REST; MySQL conserve les données; Firebase prend en charge l'authentification OTP, le temps réel et les notifications web selon la compatibilité du navigateur. Le projet couvre le cycle de développement, des besoins au déploiement.

Technologies principales : React (Vite) · Tailwind CSS · Spring Boot 3.2 · Java 17 · MySQL 8 · Firebase · Docker · Nginx · GitHub Actions